

```

# FIND-S Algorithm
# Machine Learning Practical

# Training data
training_data = [
    ['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same', 'Yes'],
    ['Sunny', 'Warm', 'High', 'Strong', 'Warm', 'Same', 'Yes'],
    ['Rainy', 'Cold', 'High', 'Strong', 'Warm', 'Change', 'No'],
    ['Sunny', 'Warm', 'High', 'Strong', 'Cool', 'Change', 'Yes']
]

# Attribute names
attributes = [
    'Sky',
    'Temperature',
    'Humidity',
    'Wind',
    'Water',
    'Forecast'
]

# FIND-S Algorithm
def find_s(data):
    # Initialize hypothesis with most specific values
    hypothesis = ['0'] * len(attributes)

    print("Initial Hypothesis:", hypothesis)
    print()

    for row in data:
        example = row[:-1]
        target = row[-1]

        # Consider only positive examples
        if target == 'Yes':
            print("Positive Example:", example)

            # If this is the first positive example
            if hypothesis == ['0'] * len(attributes):
                hypothesis = example.copy()

            else:
                # Generalize hypothesis where needed
                for i in range(len(hypothesis)):
                    if hypothesis[i] != example[i]:
                        hypothesis[i] = '?'

            print("Updated Hypothesis:", hypothesis)
            print()

    return hypothesis

# Run FIND-S
final_hypothesis = find_s(training_data)

print("-----")
print("Final Most Specific Hypothesis:")
print(final_hypothesis)
print("-----")

```

Initial Hypothesis: ['0', '0', '0', '0', '0', '0']

Positive Example: ['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same']
 Updated Hypothesis: ['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same']

Positive Example: ['Sunny', 'Warm', 'High', 'Strong', 'Warm', 'Same']
 Updated Hypothesis: ['Sunny', 'Warm', '?', 'Strong', 'Warm', 'Same']

Positive Example: ['Sunny', 'Warm', 'High', 'Strong', 'Cool', 'Change']
 Updated Hypothesis: ['Sunny', 'Warm', '?', 'Strong', '?', '?']

 Final Most Specific Hypothesis:
 ['Sunny', 'Warm', '?', 'Strong', '?', '?']

exp no 1

```

# Candidate-Elimination Algorithm
# Machine Learning Practical

import pandas as pd

# -----
# STEP 1: Create a sample CSV file
# -----

data = {
    'Sky': ['Sunny', 'Sunny', 'Rainy', 'Sunny'],
    'Temperature': ['Warm', 'Warm', 'Cold', 'Warm'],
    'Humidity': ['Normal', 'High', 'High', 'High'],
    'Wind': ['Strong', 'Strong', 'Strong', 'Strong'],
    'Water': ['Warm', 'Warm', 'Warm', 'Cool'],
    'Forecast': ['Same', 'Same', 'Change', 'Change'],
    'EnjoySport': ['Yes', 'Yes', 'No', 'Yes']
}

df = pd.DataFrame(data)

# Save data as CSV
df.to_csv('training_data.csv', index=False)

print("Training data:")
print(df)
print()

# -----
# STEP 2: Read CSV file
# -----

data = pd.read_csv('training_data.csv')

attributes = list(data.columns[:-1])
target = data.columns[-1]

X = data.iloc[:, :-1].values
Y = data.iloc[:, -1].values

print("Attributes:", attributes)
print()

# -----
# STEP 3: Initialize S and G
# -----

S = ['0'] * len(attributes)
G = [['?'] * len(attributes)]

# -----
# STEP 4: Candidate-Elimination Algorithm
# -----

for i in range(len(X)):

    example = X[i]
    label = Y[i]

    print("Example", i + 1, ":", list(example), "=>", label)

    # -----
    # Positive Example
    # -----
    if label == 'Yes':

        # Remove inconsistent hypotheses from G
        G = [
            g for g in G
            if all(g[j] == '?' or g[j] == example[j])

```

```

        for j in range(len(attributes)):
    ]

    # Generalize S
    if S == ['0'] * len(attributes):
        S = list(example)

    else:
        for j in range(len(attributes)):
            if S[j] != example[j]:
                S[j] = '?'

# -----
# Negative Example
# -----
else:

    new_G = []

    for g in G:

        # If G is already consistent with negative example,
        # specialize it
        if all(g[j] == '?' or g[j] == example[j]
               for j in range(len(attributes))):

            for j in range(len(attributes)):

                if g[j] == '?' and S[j] != example[j]:
                    new_hypothesis = g.copy()
                    new_hypothesis[j] = S[j]
                    new_G.append(new_hypothesis)

            else:
                new_G.append(g)

    G = new_G

print("S =", S)
print("G =", G)
print()

# -----
# STEP 5: Display Final Result
# -----

print("=====")
print("FINAL RESULT")
print("=====")

print("\nSpecific Boundary (S):")
print(S)

print("\nGeneral Boundary (G):")
for g in G:
    print(g)

print("\nAll hypotheses consistent with training data")
print("are bounded by S and G.")

Training data:
  Sky Temperature Humidity   Wind Water Forecast EnjoySport
0  Sunny         Warm   Normal Strong  Warm    Same      Yes
1  Sunny         Warm   High  Strong  Warm    Same      Yes
2  Rainy         Cold   High  Strong  Warm    Change     No
3  Sunny         Warm   High  Strong  Cool    Change     Yes

Attributes: ['Sky', 'Temperature', 'Humidity', 'Wind', 'Water', 'Forecast']

Example 1 : ['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same'] => Yes
S = ['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same']
G = [['?', '?', '?', '?', '?', '?']]

Example 2 : ['Sunny', 'Warm', 'High', 'Strong', 'Warm', 'Same'] => Yes
S = ['Sunny', 'Warm', '?', 'Strong', 'Warm', 'Same']
G = [['?', '?', '?', '?', '?', '?']]

Example 3 : ['Rainy', 'Cold', 'High', 'Strong', 'Warm', 'Change'] => No

```

```

S = ['Sunny', 'Warm', '?', 'Strong', 'Warm', 'Same']
G = [['Sunny', '?', '?', '?', '?', '?'], ['?', 'Warm', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'],

Example 4 : ['Sunny', 'Warm', 'High', 'Strong', 'Cool', 'Change'] => Yes
S = ['Sunny', 'Warm', '?', 'Strong', '?', '?']
G = [['Sunny', '?', '?', '?', '?', '?'], ['?', 'Warm', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?']]

=====
FINAL RESULT
=====

Specific Boundary (S):
['Sunny', 'Warm', '?', 'Strong', '?', '?']

General Boundary (G):
['Sunny', '?', '?', '?', '?', '?']
['?', 'Warm', '?', '?', '?', '?']
['?', '?', '?', '?', '?', '?']

All hypotheses consistent with training data
are bounded by S and G.

```

exp no 2

```

# ID3 Decision Tree Algorithm
# Machine Learning Practical

import pandas as pd
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt

# -----
# STEP 1: Create the dataset
# -----

data = {
    'Outlook': ['Sunny', 'Sunny', 'Overcast', 'Rain', 'Rain',
               'Rain', 'Overcast', 'Sunny', 'Sunny', 'Rain',
               'Sunny', 'Overcast', 'Overcast', 'Rain'],
    'Temperature': ['Hot', 'Hot', 'Hot', 'Mild', 'Cool',
                   'Cool', 'Cool', 'Mild', 'Cool', 'Mild',
                   'Mild', 'Mild', 'Hot', 'Mild'],
    'Humidity': ['High', 'High', 'High', 'High', 'Normal',
                'Normal', 'Normal', 'High', 'Normal', 'Normal',
                'Normal', 'High', 'Normal', 'High'],
    'Wind': ['Weak', 'Strong', 'Weak', 'Weak', 'Weak',
             'Strong', 'Strong', 'Weak', 'Weak', 'Weak',
             'Strong', 'Strong', 'Weak', 'Strong'],
    'Play': ['No', 'No', 'Yes', 'Yes', 'Yes',
             'No', 'Yes', 'No', 'Yes', 'Yes',
             'Yes', 'Yes', 'Yes', 'No']
}

df = pd.DataFrame(data)

print("Training Dataset:")
print(df)
print()

# -----
# STEP 2: Convert categorical data into numbers
# -----

X = pd.get_dummies(df.drop('Play', axis=1))
y = df['Play']

# -----
# STEP 3: Build ID3 Decision Tree
# -----

model = DecisionTreeClassifier(

```

```
criterion='entropy',
random_state=42
)

model.fit(X, y)

# -----
# STEP 4: Display Decision Tree
# -----

plt.figure(figsize=(15, 8))

plot_tree(
    model,
    feature_names=X.columns,
    class_names=model.classes_,
    filled=True,
    rounded=True
)

plt.title("Decision Tree using ID3 Algorithm")
plt.show()

# -----
# STEP 5: Classify a new sample
# -----

new_sample = pd.DataFrame({
    'Outlook': ['Sunny'],
    'Temperature': ['Cool'],
    'Humidity': ['High'],
    'Wind': ['Strong']
})

# Convert new sample using same encoding
new_sample = pd.get_dummies(new_sample)

# Make sure columns are same as training data
new_sample = new_sample.reindex(
    columns=X.columns,
    fill_value=0
)

prediction = model.predict(new_sample)

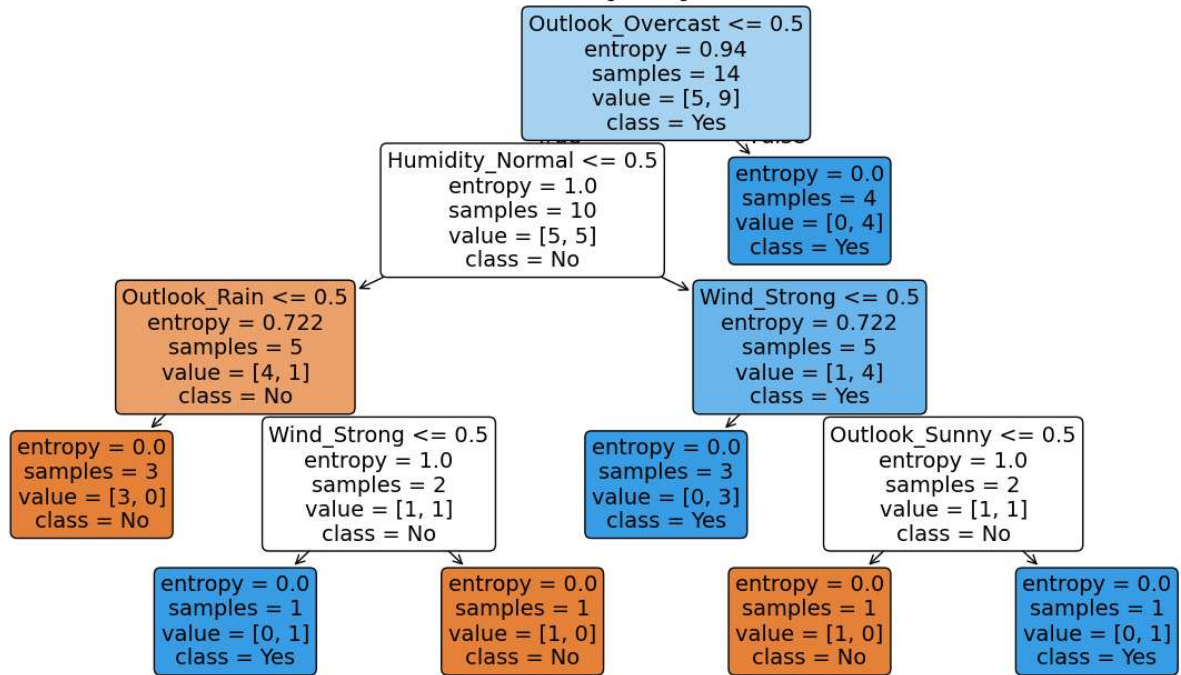
print("New Sample:")
print({
    'Outlook': 'Sunny',
    'Temperature': 'Cool',
    'Humidity': 'High',
    'Wind': 'Strong'
})

print("\nPredicted Class:", prediction[0])
```

Training Dataset:

	Outlook	Temperature	Humidity	Wind	Play
0	Sunny	Hot	High	Weak	No
1	Sunny	Hot	High	Strong	No
2	Overcast	Hot	High	Weak	Yes
3	Rain	Mild	High	Weak	Yes
4	Rain	Cool	Normal	Weak	Yes
5	Rain	Cool	Normal	Strong	No
6	Overcast	Cool	Normal	Strong	Yes
7	Sunny	Mild	High	Weak	No
8	Sunny	Cool	Normal	Weak	Yes
9	Rain	Mild	Normal	Weak	Yes
10	Sunny	Mild	Normal	Strong	Yes
11	Overcast	Mild	High	Strong	Yes
12	Overcast	Hot	Normal	Weak	Yes
13	Rain	Mild	High	Strong	No

Decision Tree using ID3 Algorithm



New Sample:

{'Outlook': 'Sunny', 'Temperature': 'Cool', 'Humidity': 'High', 'Wind': 'Strong'}

Predicted Class: No

exp no 3

```

# Artificial Neural Network using Backpropagation
# Machine Learning Practical

import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, classification_report

# -----
# STEP 1: Load Iris Dataset
# -----

iris = load_iris()

X = iris.data
y = iris.target

```

```

print("Dataset Shape:", X.shape)
print("Classes:", iris.target_names)

# -----
# STEP 2: Split Dataset into Training and Testing
# -----

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# -----
# STEP 3: Normalize the Input Data
# -----

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# -----
# STEP 4: Create ANN
# -----
# hidden_layer_sizes=(10,) means:
# One hidden layer with 10 neurons
#
# activation='relu' -> activation function
# solver='sgd' -> gradient descent
# learning_rate_init -> learning rate
# max_iter -> number of training iterations

model = MLPClassifier(
    hidden_layer_sizes=(10,),
    activation='relu',
    solver='sgd',
    learning_rate_init=0.01,
    max_iter=1000,
    random_state=42
)

# -----
# STEP 5: Train ANN using Backpropagation
# -----

model.fit(X_train, y_train)

print("\nANN training completed successfully.")

# -----
# STEP 6: Test the ANN
# -----

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

print("\nActual Values:")
print(y_test)

print("\nPredicted Values:")
print(y_pred)

print("\nAccuracy:", accuracy * 100, "%")

# -----
# STEP 7: Classification Report
# -----

```

```

print("\nClassification Report:")
print(
    classification_report(
        y_test,
        y_pred,
        target_names=iris.target_names
    )
)

# -----
# STEP 8: Test a New Sample
# -----

new_sample = np.array([[5.1, 3.5, 1.4, 0.2]])

new_sample_scaled = scaler.transform(new_sample)

prediction = model.predict(new_sample_scaled)

print("\nNew Sample:")
print(new_sample)

print("Predicted Class:", iris.target_names[prediction[0]])

```

Dataset Shape: (150, 4)
Classes: ['setosa' 'versicolor' 'virginica']

ANN training completed successfully.

Actual Values:
[0 2 1 1 0 1 0 0 2 1 2 2 2 1 0 0 0 1 1 2 0 2 1 2 2 1 1 0 2 0]

Predicted Values:
[0 2 1 1 0 1 0 0 2 1 2 2 2 1 0 0 0 1 1 2 0 2 1 2 2 2 1 0 2 0]

Accuracy: 96.66666666666667 %

Classification Report:

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	10
versicolor	1.00	0.90	0.95	10
virginica	0.91	1.00	0.95	10
accuracy			0.97	30
macro avg	0.97	0.97	0.97	30
weighted avg	0.97	0.97	0.97	30

New Sample:
[[5.1 3.5 1.4 0.2]]
Predicted Class: setosa

exp no 4

```

# K-Nearest Neighbours (K-NN) Classification
# Google Colab Program

# Step 1: Import required libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Step 2: Load the Iris dataset
iris = load_iris()

X = iris.data
y = iris.target

# Display dataset
print("Dataset Shape:", X.shape)

```

```

print("Dataset Shape: ", X.shape)
print("\nFirst 5 rows of dataset:")
print(pd.DataFrame(X, columns=iris.feature_names).head())

# Step 3: Split the dataset into training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("\nTraining samples:", len(X_train))
print("Testing samples:", len(X_test))

# Step 4: Feature scaling
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Step 5: Create K-NN classifier
k = 5

knn = KNeighborsClassifier(n_neighbors=k)

# Step 6: Train the model
knn.fit(X_train, y_train)

# Step 7: Make predictions
y_pred = knn.predict(X_test)

# Step 8: Display predictions
print("\nActual values: ", y_test)
print("Predicted values:", y_pred)

# Step 9: Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)

print("\nK-NN Classification Accuracy:", accuracy * 100, "%")

# Step 10: Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix:")
print(cm)

# Step 11: Classification Report
print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=iris.target_names))

# Step 12: Predict a new sample
# Features: Sepal Length, Sepal Width, Petal Length, Petal Width
new_sample = [[5.1, 3.5, 1.4, 0.2]]

# Scale the new sample
new_sample_scaled = scaler.transform(new_sample)

# Predict
prediction = knn.predict(new_sample_scaled)

print("\nNew Sample:", new_sample)
print("Predicted Class:", iris.target_names[prediction[0]])

```

Dataset Shape: (150, 4)

First 5 rows of dataset:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2

Training samples: 120

Testing samples: 30

Actual values: [0 2 1 1 0 1 0 0 2 1 2 2 2 1 0 0 0 1 1 2 0 2 1 2 2 1 1 0 2 0]

Predicted values: [0 2 1 1 0 1 0 0 2 1 2 2 2 1 0 0 0 1 1 1 0 2 1 1 2 1 1 0 2 0]

K-NN Classification Accuracy: 93.33333333333333 %

Confusion Matrix:

```
[[10  0  0]
 [ 0 10  0]
 [ 0  2  8]]
```

Classification Report:

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	10
versicolor	0.83	1.00	0.91	10
virginica	1.00	0.80	0.89	10
accuracy			0.93	30
macro avg	0.94	0.93	0.93	30
weighted avg	0.94	0.93	0.93	30

New Sample: [[5.1, 3.5, 1.4, 0.2]]

Predicted Class: setosa

```
# Naïve Bayes Classification
# Google Colab

# Step 1: Import libraries
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, confusion_matrix, ConfusionMatrixDisplay

# Step 2: Load the Iris dataset
iris = load_iris()

X = iris.data
y = iris.target

# Step 3: Split the dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))

# Step 4: Create Naïve Bayes classifier
model = GaussianNB()

# Step 5: Train the model
model.fit(X_train, y_train)

# Step 6: Predict the test data
y_pred = model.predict(X_test)

# Step 7: Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)

print("\nActual Values: ", y_test)
print("Predicted Values:", y_pred)

print("\nAccuracy:", accuracy * 100, "%")

# Step 8: Generate confusion matrix
cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix:")
print(cm)

# Step 9: Display confusion matrix graphically
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=iris.target_names
)
```

```

disp.plot()
plt.title("Naïve Bayes - Confusion Matrix")
plt.show()

```

Training samples: 120

Testing samples: 30

Actual Values: [0 2 1 1 0 1 0 0 2 1 2 2 2 1 0 0 0 1 1 2 0 2 1 2 2 1 1 0 2 0]

Predicted Values: [0 2 1 1 0 1 0 0 2 1 2 2 2 1 0 0 0 1 1 2 0 2 1 2 2 2 1 0 2 0]

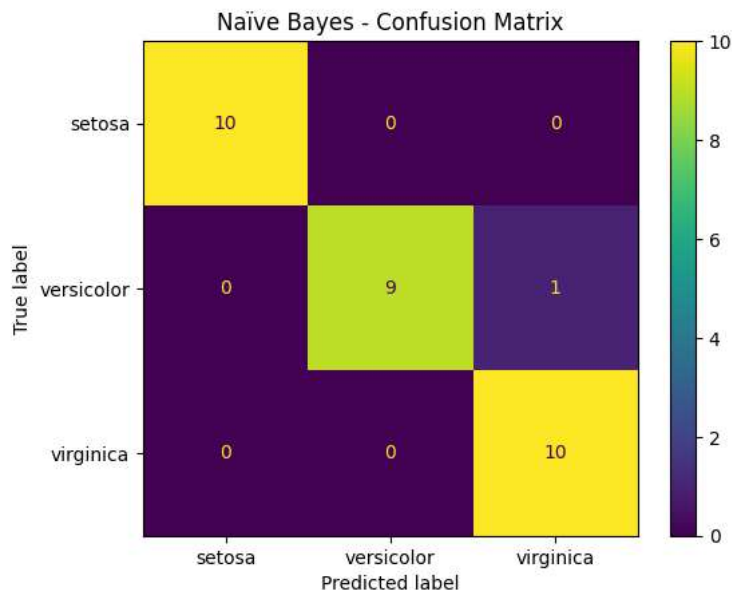
Accuracy: 96.66666666666667 %

Confusion Matrix:

```

[[10  0  0]
 [ 0  9  1]
 [ 0  0 10]]

```



EXP 6

```

# Logistic Regression (LR) Classification
# Google Colab

# Step 1: Import required libraries
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, ConfusionMatrixDisplay

# Step 2: Load the Iris dataset
iris = load_iris()

X = iris.data
y = iris.target

# Step 3: Split dataset into training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# Step 4: Feature scaling
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Step 5: Create Logistic Regression model

```

```

model = LogisticRegression(max_iter=200)

# Step 6: Train the model
model.fit(X_train, y_train)

# Step 7: Make predictions
y_pred = model.predict(X_test)

# Step 8: Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)

print("Actual Values: ", y_test)
print("Predicted Values:", y_pred)

print("\nLogistic Regression Accuracy:",
      accuracy * 100, "%")

# Step 9: Generate confusion matrix
cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix:")
print(cm)

# Step 10: Display confusion matrix
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=iris.target_names
)

disp.plot()
plt.title("Logistic Regression - Confusion Matrix")
plt.show()

```

```

Actual Values:  [0 2 1 1 0 1 0 0 2 1 2 2 2 1 0 0 0 1 1 2 0 2 1 2 2 1 1 0 2 0]
Predicted Values: [0 2 1 1 0 1 0 0 2 1 2 2 2 1 0 0 0 1 1 2 0 2 1 1 2 2 1 0 2 0]

```

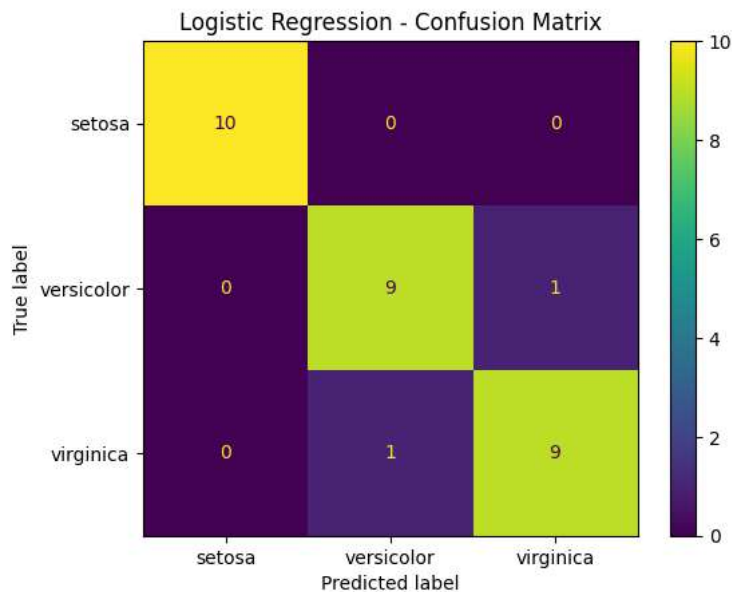
Logistic Regression Accuracy: 93.33333333333333 %

Confusion Matrix:

```

[[10  0  0]
 [ 0  9  1]
 [ 0  1  9]]

```



Double-click (or enter) to edit

EXP 7

```

# Linear Regression in Python
# Google Colab

# Step 1: Import required libraries

```

```
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Step 2: Create sample dataset
# X = Hours studied
# y = Marks obtained

X = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10]).reshape(-1, 1)
y = np.array([20, 25, 35, 40, 50, 55, 65, 70, 80, 90])

# Step 3: Split dataset into training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

# Step 4: Create Linear Regression model
model = LinearRegression()

# Step 5: Train the model
model.fit(X_train, y_train)

# Step 6: Make predictions
y_pred = model.predict(X_test)

# Step 7: Display actual and predicted values
print("Actual Values: ", y_test)
print("Predicted Values:", np.round(y_pred, 2))

# Step 8: Calculate Mean Squared Error
mse = mean_squared_error(y_test, y_pred)

# Step 9: Calculate R2 Score
r2 = r2_score(y_test, y_pred)

print("\nMean Squared Error (MSE):", round(mse, 2))
print("R2 Score:", round(r2, 2))

# Step 10: Display model equation
print("\nRegression Equation:")
print("Marks =", round(model.coef_[0], 2),
      "* Hours +", round(model.intercept_, 2))

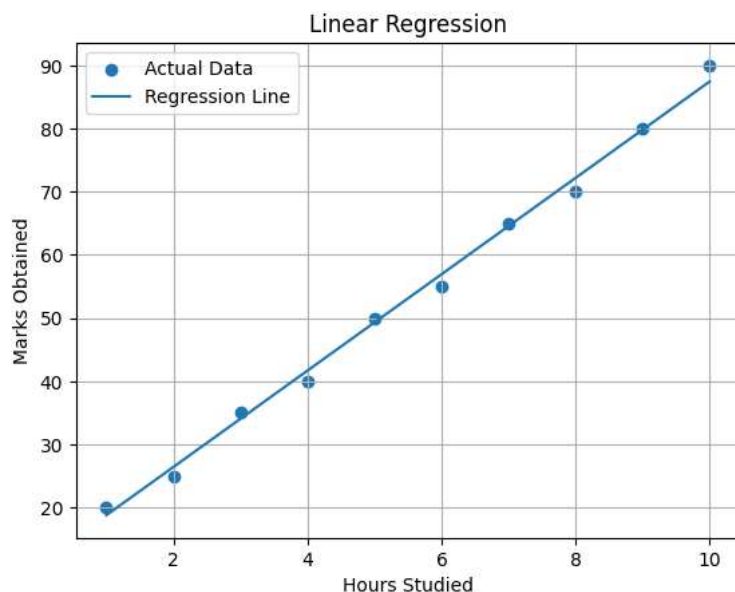
# Step 11: Plot regression line
plt.scatter(X, y, label="Actual Data")
plt.plot(X, model.predict(X), label="Regression Line")

plt.xlabel("Hours Studied")
plt.ylabel("Marks Obtained")
plt.title("Linear Regression")
plt.legend()
plt.grid(True)
plt.show()
```

Actual Values: [80 25]
 Predicted Values: [79.83 26.42]

Mean Squared Error (MSE): 1.03
 R2 Score: 1.0

Regression Equation:
 Marks = 7.63 * Hours + 11.16



EXP 8

```
# Compare Linear Regression and Polynomial Regression
# Google Colab

# Step 1: Import libraries
import numpy as np
import matplotlib.pyplot as plt

from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.metrics import mean_squared_error, r2_score

# Step 2: Create dataset
# X = Hours studied
# y = Marks obtained

X = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10]).reshape(-1, 1)

y = np.array([20, 25, 35, 45, 60, 75, 80, 85, 90, 95])

# Step 3: Linear Regression
linear_model = LinearRegression()
linear_model.fit(X, y)

y_linear = linear_model.predict(X)

# Step 4: Polynomial Regression
poly = PolynomialFeatures(degree=2)

X_poly = poly.fit_transform(X)

poly_model = LinearRegression()
poly_model.fit(X_poly, y)

y_poly = poly_model.predict(X_poly)

# Step 5: Calculate performance
linear_mse = mean_squared_error(y, y_linear)
linear_r2 = r2_score(y, y_linear)

poly_mse = mean_squared_error(y, y_poly)
poly_r2 = r2_score(y, y_poly)
```

```

# Step 6: Display results
print("==== LINEAR REGRESSION =====")
print("Mean Squared Error:", round(linear_mse, 2))
print("R2 Score:", round(linear_r2, 2))

print("\n==== POLYNOMIAL REGRESSION =====")
print("Mean Squared Error:", round(poly_mse, 2))
print("R2 Score:", round(poly_r2, 2))

# Step 7: Compare results
print("\n==== COMPARISON =====")

if poly_r2 > linear_r2:
    print("Polynomial Regression gives better R2 Score.")
else:
    print("Linear Regression gives better R2 Score.")

if poly_mse < linear_mse:
    print("Polynomial Regression gives lower MSE.")
else:
    print("Linear Regression gives lower MSE.")

# Step 8: Plot both regression models
X_plot = np.linspace(1, 10, 100).reshape(-1, 1)

y_linear_plot = linear_model.predict(X_plot)

X_plot_poly = poly.transform(X_plot)
y_poly_plot = poly_model.predict(X_plot_poly)

plt.scatter(X, y, label="Actual Data")
plt.plot(X_plot, y_linear_plot, label="Linear Regression")
plt.plot(X_plot, y_poly_plot, label="Polynomial Regression")

plt.xlabel("Hours Studied")
plt.ylabel("Marks Obtained")
plt.title("Linear vs Polynomial Regression")
plt.legend()
plt.grid(True)
plt.show()

```

```

==== LINEAR REGRESSION =====
Mean Squared Error: 22.18
R2 Score: 0.97

```

```

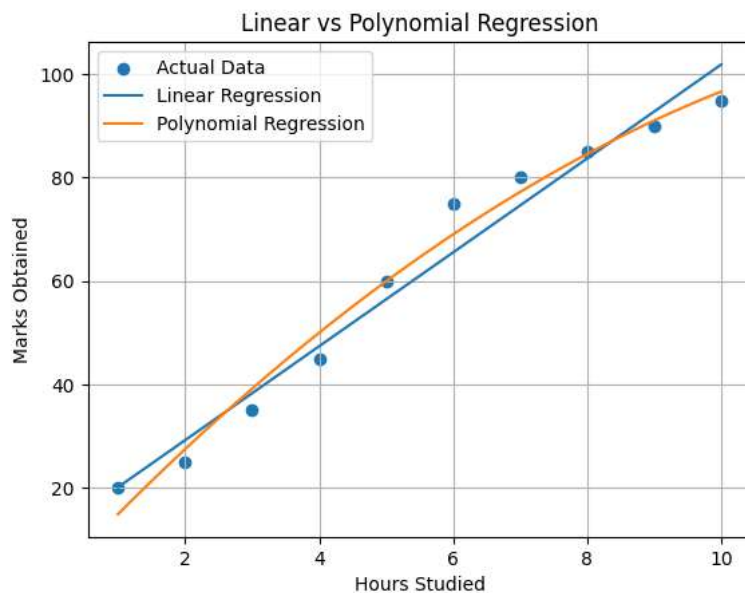
==== POLYNOMIAL REGRESSION =====
Mean Squared Error: 12.16
R2 Score: 0.98

```

```

==== COMPARISON =====
Polynomial Regression gives better R2 Score.
Polynomial Regression gives lower MSE.

```



EXP 9

```

# Expectation-Maximization (EM) Algorithm
# Using Gaussian Mixture Model (GMM)
# Google Colab

# Step 1: Import required libraries
import numpy as np
import matplotlib.pyplot as plt

from sklearn.mixture import GaussianMixture
from sklearn.datasets import make_blobs

# Step 2: Create sample dataset
X, y = make_blobs(
    n_samples=300,
    centers=3,
    cluster_std=1.0,
    random_state=42
)

# Step 3: Create Gaussian Mixture Model
# n_components = number of clusters
gmm = GaussianMixture(
    n_components=3,
    random_state=42
)

# Step 4: Apply EM algorithm
gmm.fit(X)

# Step 5: Predict cluster labels
labels = gmm.predict(X)

# Step 6: Display results
print("EM Algorithm Results")
print("-----")

print("Number of clusters:", gmm.n_components)
print("Number of iterations:", gmm.n_iter_)
print("Converged:", gmm.converged_)

print("\nCluster Means:")
print(gmm.means_)

print("\nCluster Weights:")
print(gmm.weights_)

# Step 7: Plot the clusters
plt.figure(figsize=(8, 6))

plt.scatter(
    X[:, 0],
    X[:, 1],
    c=labels,
    s=40
)

plt.scatter(
    gmm.means_[ :, 0],
    gmm.means_[ :, 1],
    marker='X',
    s=200,
    label='Cluster Centers'
)

plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("Expectation-Maximization Algorithm using GMM")
plt.legend()
plt.grid(True)
plt.show()

```

EM Algorithm Results

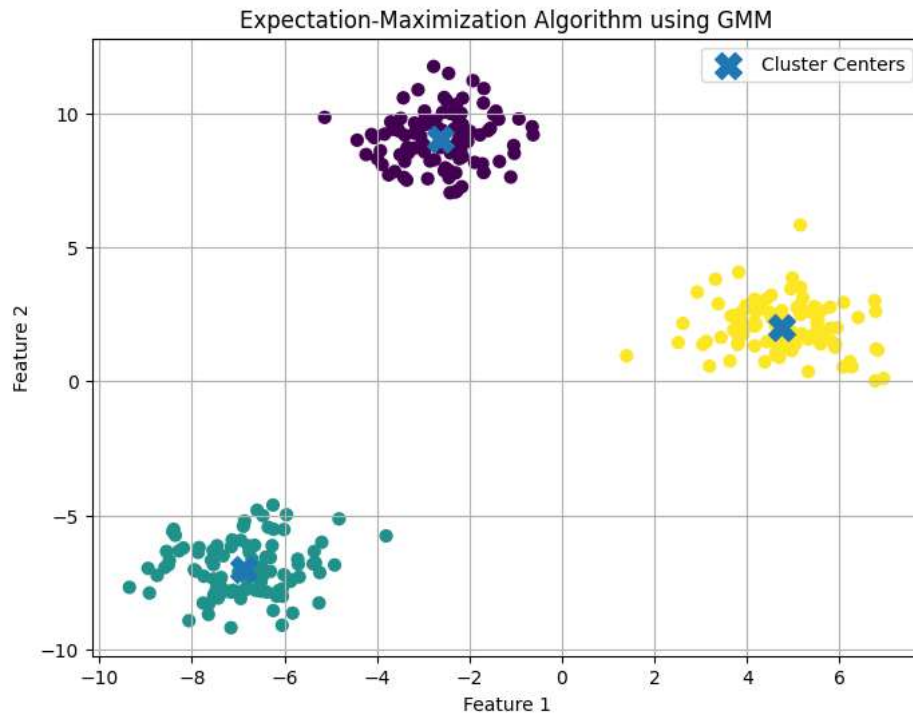
 Number of clusters: 3
 Number of iterations: 2
 Converged: True

Cluster Means:

```
[[-2.63323268  9.04356978]
 [-6.88387179 -6.98398415]
 [ 4.74710337  2.01059427]]
```

Cluster Weights:

```
[0.33333333 0.33333333 0.33333333]
```



EXP 10

```
# Credit Score Classification using Machine Learning
# Google Colab

# Step 1: Import required libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Step 2: Create a sample credit score dataset
data = {
    'Income': [25000, 30000, 45000, 50000, 60000, 70000, 80000, 90000,
               35000, 40000, 55000, 65000, 75000, 85000, 95000],
    'Age': [22, 25, 30, 35, 40, 45, 50, 55,
            28, 32, 38, 42, 48, 52, 60],
    'Loan_Amount': [20000, 25000, 30000, 35000, 30000, 25000, 20000, 15000,
                    40000, 35000, 30000, 25000, 20000, 15000, 10000],
    'Credit_History': [1, 1, 1, 1, 1, 1, 1, 1,
                       0, 0, 1, 1, 1, 1, 1],
    'Credit_Score': [
        'Poor', 'Poor', 'Average', 'Average', 'Good',
        'Good', 'Good', 'Good', 'Poor', 'Poor',
        'Average', 'Good', 'Good', 'Good', 'Good']
}
```

```

    ]
}

df = pd.DataFrame(data)

# Step 3: Display dataset
print("Credit Score Dataset:")
print(df)

# Step 4: Convert target labels into numbers
df['Credit_Score'] = df['Credit_Score'].map({
    'Poor': 0,
    'Average': 1,
    'Good': 2
})

# Step 5: Separate features and target
X = df[['Income', 'Age', 'Loan_Amount', 'Credit_History']]
y = df['Credit_Score']

# Step 6: Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# Step 7: Feature scaling
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Step 8: Create Random Forest model
model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

# Step 9: Train the model
model.fit(X_train, y_train)

# Step 10: Make predictions
y_pred = model.predict(X_test)

# Step 11: Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)

print("\nActual Values: ", y_test.values)
print("Predicted Values:", y_pred)

print("\nAccuracy:",
      round(accuracy * 100, 2), "%")

# Step 12: Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix:")
print(cm)

# Step 13: Classification Report
print("\nClassification Report:")

print(classification_report(
    y_test,
    y_pred,
    labels=[0, 1, 2],
    target_names=['Poor', 'Average', 'Good'],
    zero_division=0
))

# Step 14: Predict a new customer's credit score
new_customer = pd.DataFrame({
    'Income': [65000],
    'Age': [40],
    'Loan_Amount': [20000],

```

```

    'Credit_History': [1]
})

new_customer_scaled = scaler.transform(new_customer)

prediction = model.predict(new_customer_scaled)

score_names = {
    0: 'Poor',
    1: 'Average',
    2: 'Good'
}

print("New Customer Credit Score:",
      score_names[prediction[0]])

```

Credit Score Dataset:

	Income	Age	Loan_Amount	Credit_History	Credit_Score
0	25000	22	20000	1	Poor
1	30000	25	25000	1	Poor
2	45000	30	30000	1	Average
3	50000	35	35000	1	Average
4	60000	40	30000	1	Good
5	70000	45	25000	1	Good
6	80000	50	20000	1	Good
7	90000	55	15000	1	Good
8	35000	28	40000	0	Poor
9	40000	32	35000	0	Poor
10	55000	38	30000	1	Average
11	65000	42	25000	1	Good
12	75000	48	20000	1	Good
13	85000	52	15000	1	Good
14	95000	60	10000	1	Good

Actual Values: [0 2 1]

Predicted Values: [1 2 1]

Accuracy: 66.67 %

Confusion Matrix:

```

[[0 1 0]
 [0 1 0]
 [0 0 1]]

```

Classification Report:

	precision	recall	f1-score	support
Poor	0.00	0.00	0.00	1
Average	0.50	1.00	0.67	1
Good	1.00	1.00	1.00	1
accuracy			0.67	3
macro avg	0.50	0.67	0.56	3
weighted avg	0.50	0.67	0.56	3

New Customer Credit Score: Good

EXP 11

```

# Iris Flower Classification using KNN

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

# Load dataset
iris = load_iris()
X = iris.data
y = iris.target

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Scale data
scaler = StandardScaler()

```

```

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# KNN model
model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Results
print("Actual Values: ", y_test)
print("Predicted Values:", y_pred)
print("\nAccuracy:", accuracy_score(y_test, y_pred) * 100, "%")
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

Actual Values:  [0 2 1 1 0 1 0 0 2 1 2 2 2 1 0 0 0 1 1 2 0 2 1 2 2 1 1 0 2 0]
Predicted Values: [0 2 1 1 0 1 0 0 2 1 2 2 2 1 0 0 0 1 1 1 0 2 1 1 2 1 1 0 2 0]

Accuracy: 93.33333333333333 %

Confusion Matrix:
[[10  0  0]
 [ 0 10  0]
 [ 0  2  8]]

```

EXP 12

```

# Car Price Prediction using Linear Regression

import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Create dataset
data = {
    "Year": [2015, 2016, 2017, 2018, 2019, 2020, 2021, 2022, 2017, 2019],
    "Kilometers": [80000, 70000, 60000, 50000, 40000, 30000, 20000, 10000, 55000, 35000],
    "Engine": [1200, 1200, 1500, 1500, 1600, 1600, 1800, 1800, 1400, 1600],
    "Price": [350000, 400000, 450000, 500000, 600000, 700000, 850000, 1000000, 480000, 650000]
}

df = pd.DataFrame(data)

# Features and target
X = df[["Year", "Kilometers", "Engine"]]
y = df["Price"]

# Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Train model
model = LinearRegression()
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

print("Actual Prices: ", y_test.values)
print("Predicted Prices:", np.round(y_pred, 2))

print("\nMSE:", round(mean_squared_error(y_test, y_pred), 2))
print("R2 Score:", round(r2_score(y_test, y_pred), 2))

# New car prediction
new_car = [[2021, 25000, 1600]]
print("\nPredicted Price for New Car:",
      round(model.predict(new_car)[0], 2))

Actual Prices:  [480000 400000]
Predicted Prices: [473094.69 404272.52]

```

```
MSE: 32968867.51
R2 Score: 0.98
```

```
Predicted Price for New Car: 866166.28
/usr/local/lib/python3.13/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but
warnings.warn(
```

EXP 13

```
# House Price Prediction using Linear Regression

import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Dataset
data = {
    "Area": [800, 1000, 1200, 1500, 1800, 2000, 2200, 2500, 2800, 3000],
    "Bedrooms": [2, 2, 3, 3, 3, 4, 4, 4, 5, 5],
    "Age": [20, 15, 12, 10, 8, 6, 5, 4, 3, 2],
    "Price": [2000000, 2500000, 3000000, 3800000, 4500000,
              5200000, 5800000, 6500000, 7500000, 8200000]
}

df = pd.DataFrame(data)

X = df[["Area", "Bedrooms", "Age"]]
y = df["Price"]

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Model
model = LinearRegression()
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

print("Actual Prices: ", y_test.values)
print("Predicted Prices:", np.round(y_pred, 2))

print("\nMSE:", round(mean_squared_error(y_test, y_pred), 2))
print("R2 Score:", round(r2_score(y_test, y_pred), 2))

# New house prediction
new_house = [[2000, 4, 5]]
print("\nPredicted House Price:",
      round(model.predict(new_house)[0], 2))
```

```
Actual Prices: [7500000 2500000]
Predicted Prices: [7654962.  2304600.42]
```

```
MSE: 31097107200.98
R2 Score: 1.0
```

```
Predicted House Price: 5195919.62
/usr/local/lib/python3.13/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but
warnings.warn(
```

ECP 14

```
# Iris Flower Classification using Naive Bayes

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, confusion_matrix

# Load dataset
```

```

iris = load_iris()
X = iris.data
y = iris.target

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Naive Bayes model
model = GaussianNB()
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Results
print("Actual Values: ", y_test)
print("Predicted Values:", y_pred)

print("\nAccuracy:", accuracy_score(y_test, y_pred) * 100, "%")

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

Actual Values:  [0 2 1 1 0 1 0 0 2 1 2 2 2 1 0 0 0 1 1 2 0 2 1 2 2 1 1 0 2 0]
Predicted Values: [0 2 1 1 0 1 0 0 2 1 2 2 2 1 0 0 0 1 1 2 0 2 1 2 2 2 1 0 2 0]

Accuracy: 96.66666666666667 %

Confusion Matrix:
[[10  0  0]
 [ 0  9  1]
 [ 0  0 10]]

```

EXP 15

```

# Comparison of Classification Algorithms

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# Load dataset
iris = load_iris()
X = iris.data
y = iris.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Scaling for KNN and Logistic Regression
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Models
models = {
    "KNN": KNeighborsClassifier(n_neighbors=5),
    "Logistic Regression": LogisticRegression(max_iter=200),
    "Naive Bayes": GaussianNB(),
    "Decision Tree": DecisionTreeClassifier(random_state=42)
}

print("Classification Algorithm Performance")
print("-----")

for name, model in models.items():

```

```

if name in ["KNN", "Logistic Regression"]:
    model.fit(X_train_scaled, y_train)
    prediction = model.predict(X_test_scaled)
else:
    model.fit(X_train, y_train)
    prediction = model.predict(X_test)

accuracy = accuracy_score(y_test, prediction)

print(name, ":", round(accuracy * 100, 2), "%")

```

Classification Algorithm Performance

```

-----
KNN : 93.33 %
Logistic Regression : 93.33 %
Naive Bayes : 96.67 %
Decision Tree : 93.33 %

```

EXP16

```

# Mobile Price Prediction using Random Forest
# Google Colab

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

# Step 1: Create dataset
data = {
    "RAM": [
        2, 2, 3, 3, 4, 4, 4, 6, 6, 6,
        8, 8, 8, 8, 12, 12, 12, 12, 16, 16
    ],
    "Storage": [
        32, 64, 32, 64, 64, 128, 128, 128, 256, 256,
        128, 256, 256, 512, 256, 512, 512, 1024, 512, 1024
    ],
    "Battery": [
        3000, 3200, 3500, 3600, 4000, 4200, 4500, 4500, 5000, 5000,
        5000, 5000, 5200, 5500, 5000, 5500, 6000, 6000, 6000, 6500
    ],
    "Camera": [
        8, 12, 12, 16, 16, 20, 24, 32, 32, 48,
        48, 50, 64, 64, 64, 108, 108, 200, 108, 200
    ],
    # 0 = Low, 1 = Medium, 2 = High, 3 = Premium
    "Price_Range": [
        0, 0, 0, 0, 0,
        1, 1, 1, 1, 1,
        2, 2, 2, 2, 2,
        3, 3, 3, 3, 3
    ]
}

# Step 2: Convert to DataFrame
df = pd.DataFrame(data)

print("Mobile Price Dataset:")
print(df)

# Step 3: Separate input and output
X = df[["RAM", "Storage", "Battery", "Camera"]]
y = df["Price_Range"]

# Step 4: Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.25,
    random_state=42,
    stratify=y
)

```

```

# Step 5: Create Random Forest model
model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

# Step 6: Train the model
model.fit(X_train, y_train)

# Step 7: Predict test data
y_pred = model.predict(X_test)

# Step 8: Display results
print("\nActual Values:")
print(y_test.values)

print("\nPredicted Values:")
print(y_pred)

# Step 9: Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)

print("\nAccuracy:", round(accuracy * 100, 2), "%")

# Step 10: Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix:")
print(cm)

# Step 11: Predict a new mobile
new_mobile = pd.DataFrame({
    "RAM": [8],
    "Storage": [256],
    "Battery": [5000],
    "Camera": [50]
})

prediction = model.predict(new_mobile)

price_ranges = {
    0: "Low",
    1: "Medium",
    2: "High",
    3: "Premium"
}

print("\nNew Mobile Specifications:")
print(new_mobile)

print("\nPredicted Price Range:",
      price_ranges[prediction[0]])

```

Mobile Price Dataset:

	RAM	Storage	Battery	Camera	Price_Range
0	2	32	3000	8	0
1	2	64	3200	12	0
2	3	32	3500	12	0
3	3	64	3600	16	0
4	4	64	4000	16	0
5	4	128	4200	20	1
6	4	128	4500	24	1
7	6	128	4500	32	1
8	6	256	5000	32	1
9	6	256	5000	48	1
10	8	128	5000	48	2
11	8	256	5000	50	2
12	8	256	5200	64	2
13	8	512	5500	64	2
14	12	256	5000	64	2
15	12	512	5500	108	3
16	12	512	6000	108	3
17	12	1024	6000	200	3
18	16	512	6000	108	3
19	16	1024	6500	200	3

Actual Values:
[0 3 2 2 1]

Predicted Values:

[0 3 2 2 1]

Accuracy: 100.0 %

Confusion Matrix:

[[1 0 0 0]

[0 1 0 0]

[0 0 2 0]

[0 0 0 1]]

New Mobile Specifications:

	RAM	Storage	Battery	Camera
0	8	256	5000	50

Predicted Price Range: High

EXP17

```
# Perceptron-based Iris Classification

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Perceptron
from sklearn.metrics import accuracy_score, confusion_matrix

# Load Iris dataset
iris = load_iris()

X = iris.data
y = iris.target

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Scale features
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Perceptron model
model = Perceptron(
    max_iter=1000,
    random_state=42
)

# Train
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Results
print("Actual Values: ", y_test)
print("Predicted Values:", y_pred)

print("\nAccuracy:", accuracy_score(y_test, y_pred) * 100, "%")

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

Actual Values: [0 2 1 1 0 1 0 0 2 1 2 2 2 1 0 0 0 1 1 2 0 2 1 2 2 1 1 0 2 0]
 Predicted Values: [0 2 1 1 0 2 0 0 2 1 2 2 2 2 0 0 0 1 1 2 0 2 1 1 2 2 1 0 2 0]

Accuracy: 86.66666666666667 %

Confusion Matrix:

[[10 0 0]

[0 7 3]

[0 1 9]]

Double-click (or enter) to edit

EXP18

```

# Perceptron-based Iris Classification

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Perceptron
from sklearn.metrics import accuracy_score, confusion_matrix

# Load Iris dataset
iris = load_iris()

X = iris.data
y = iris.target

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Scale features
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Perceptron model
model = Perceptron(
    max_iter=1000,
    random_state=42
)

# Train
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Results
print("Actual Values: ", y_test)
print("Predicted Values:", y_pred)

print("\nAccuracy:", accuracy_score(y_test, y_pred) * 100, "%")

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

Actual Values:   [0 2 1 1 0 1 0 0 2 1 2 2 2 1 0 0 0 1 1 2 0 2 1 2 2 1 1 0 2 0]
Predicted Values: [0 2 1 1 0 2 0 0 2 1 2 2 2 2 0 0 0 1 1 2 0 2 1 1 2 2 1 0 2 0]

Accuracy: 86.66666666666667 %

Confusion Matrix:
[[10  0  0]
 [ 0  7  3]
 [ 0  1  9]]

```

EXP19

```

# Future Sales Prediction using Linear Regression

import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

# Month numbers
months = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]).reshape(-1, 1)

# Sales values
sales = np.array([
    120, 135, 150, 160, 175, 190,
    210, 225, 240, 260, 280, 300
])

# Create model

```

```
model = LinearRegression()

# Train model
model.fit(months, sales)

# Predict future sales
future_months = np.array([13, 14, 15, 16, 17, 18]).reshape(-1, 1)

future_sales = model.predict(future_months)

print("Future Sales Prediction")
print("-----")

for month, sale in zip(future_months.flatten(), future_sales):
    print("Month", month, ":", round(sale, 2))

# Plot
plt.scatter(months, sales, label="Actual Sales")
plt.plot(months, model.predict(months), label="Regression Line")
plt.plot(future_months, future_sales, "o--",
         label="Future Prediction")

plt.xlabel("Month")
plt.ylabel("Sales")
plt.title("Future Sales Prediction")
plt.legend()
plt.grid(True)
```

```
Future Sales Prediction
-----
Month 13 : 308.86
Month 14 : 325.03
Month 15 : 341.21
Month 16 : 357.38
Month 17 : 373.55
```